Performance Innovative Task

Algorithm Discussion and Real Life Application of the Traveling Salesperson Problem

Discuss your assigned algorithm and use it to solve one real-life application of the Traveling Salesperson Problem (TSP). Present your work through a 10–15-minute audio-video presentation that includes an actual run of your Python implementation using your application data.

The TSP requires a route that visits each required location exactly once and returns to the starting location while minimizing total distance, travel time, or cost.

1 Discussion of the assigned algorithm

- Identify and explain your assigned algorithm and describe its procedure step by step.
- Discuss its advantages and limitations. Explain whether it guarantees an optimal solution or provides an approximate solution.

2 Problem and objective

- Describe one real-life situation that can be modeled as a TSP, such as delivery routing, tourist itinerary planning, or inspection visits. Specify the place or area here in Cagayan de Oro City you wanted to work with.
- Identify the starting location, all required locations, relevant assumptions, and constraints.
- Clearly state the quantity to minimize: total distance, travel time, or transportation cost.

3 Methodology and graph model

- Represent the problem using a weighted graph. Define and label each vertex and explain the location it represents. Use Google maps.
- Define the edges and explain the travel connections they represent. State whether travel is directed or undirected.
- Specify edge weights and units. Include a labeled graph and a table or matrix of weights. Identify data sources and clearly distinguish actual data from assumed data.
- Explain how your assigned algorithm is applied to your graph and show the key steps and calculations.

Python Implementation and Presentation

4 Python implementation

- Implement your assigned algorithm in Python using the same data shown in your real-life problem and graph model.
- Include a screen recording of an actual code execution in your audio-video presentation. Show the input data, run the code, and display the output. Showing code or output screenshots alone is insufficient.
- Explain the main code components and how they implement the assigned algorithm. Make the code and output readable on screen.
- Display the resulting route and its total distance, travel time, or cost, with units. Verify that every required location is visited exactly once before returning to the starting location. Recalculate the total from the edge weights to check the output.

5 Solution or conclusion

- Interpret the results and explain how the resulting route addresses the problem and meets the objective.
- State whether the route is proven optimal or is an approximate solution. Do not claim optimality without supporting evidence.
- Discuss limitations and possible improvements. You may use additional references. Cite sources used for the algorithm, code, and application data.

Audio video presentation duration

Required duration: 10–15 minutes, including the Python execution demonstration. The following allocation is a guide; you may adjust it while keeping the full presentation within the required duration.

Presentation component	Suggested time
Introduction, problem, and objective	2–3 minutes
Assigned algorithm discussion	2–3 minutes
Graph model, data, and methodology	2–3 minutes
Python execution and explanation of results	3–4 minutes
Conclusion and references	1–2 minutes

Use clear narration, readable slides, and visible code and output. Keep the discussion organized and ensure that the graph, data, Python inputs, and reported results are consistent.

Required submission materials

Submit the audio-video presentation, the Python source file or notebook, and the dataset used in the implementation. If the data are embedded in the code, identify the relevant section clearly. Include references in the presentation.

Assessment Rubric

Total possible score: 100 points. The rate for each criterion will be using the performance scale below, then multiply its maximum points by the rating factor. Add the criterion scores to obtain the final score.

Criterion	Points	Evidence expected
Assigned algorithm	20	Accurate explanation, clear procedure, appropriate advantages and limitations, and correct statement about optimality.
Problem and objective	10	Relevant TSP application with clear locations, starting point, assumptions, constraints, and measurable objective.
Graph model and data	15	Clearly defined vertices and edges; correct direction and weights; labeled graph and matching data table or matrix; units and data sources identified.
Methodology and Python implementation	25	Assigned algorithm correctly applied and implemented; actual code execution shown using the application data; inputs, main code components, and output explained.
Solution and conclusion	20	Valid closed tour; total correctly calculated and checked; results interpreted against the objective; optimality claims supported; limitations discussed.
Presentation quality and duration	5	Clear narration, logical organization, readable visuals and code, and total duration of 10–15 minutes.
Submission and references	5	Video, runnable Python file or notebook, and identifiable dataset submitted; algorithm, code, and data sources credited.
Total	100	

Performance scale

Rating	Factor	Descriptor
Excellent	1.00	All expected evidence is present, accurate, clear, and well supported.
Proficient	0.75	Most expected evidence is present and correct; minor errors or omissions do not materially affect the work.
Developing	0.50	Some expected evidence is present, but important omissions or errors weaken the work.
Beginning	0.25	Minimal evidence is present; major errors or omissions prevent a clear or reliable demonstration.
No evidence	0.00	The criterion is absent or cannot be assessed.

Example: A Proficient rating for Python implementation earns 25 × 0.75 = 18.75 points. Assess each criterion independently; presentation quality does not substitute for mathematical or programming correctness.